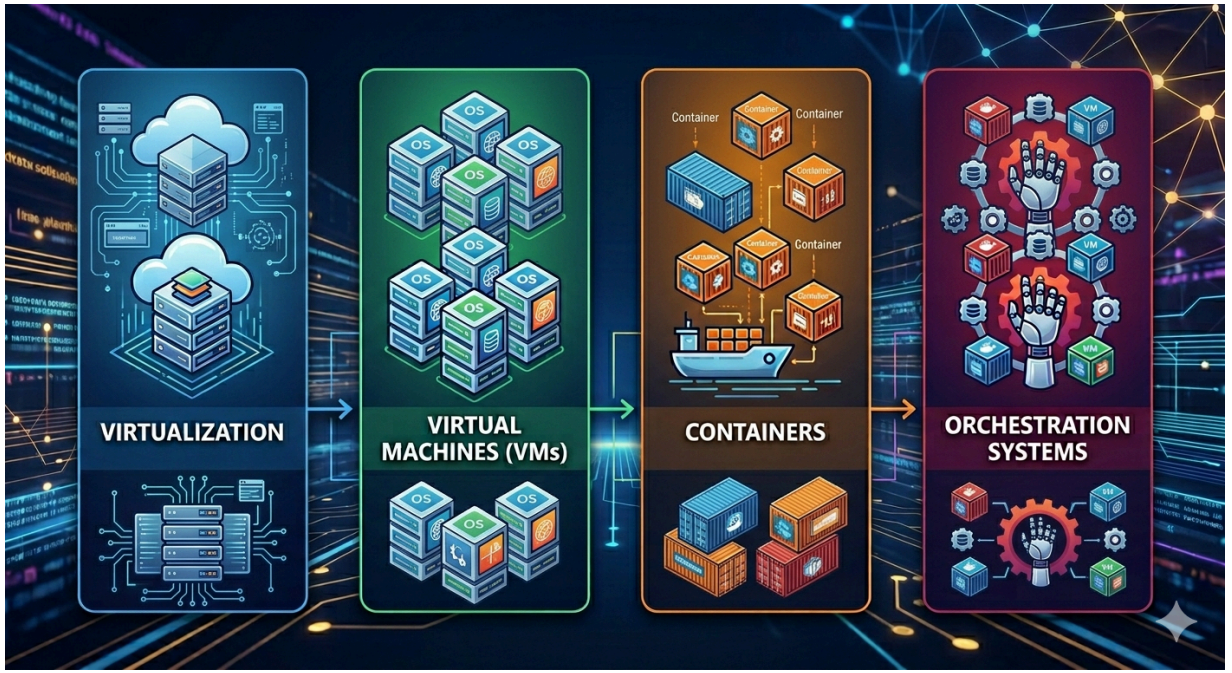
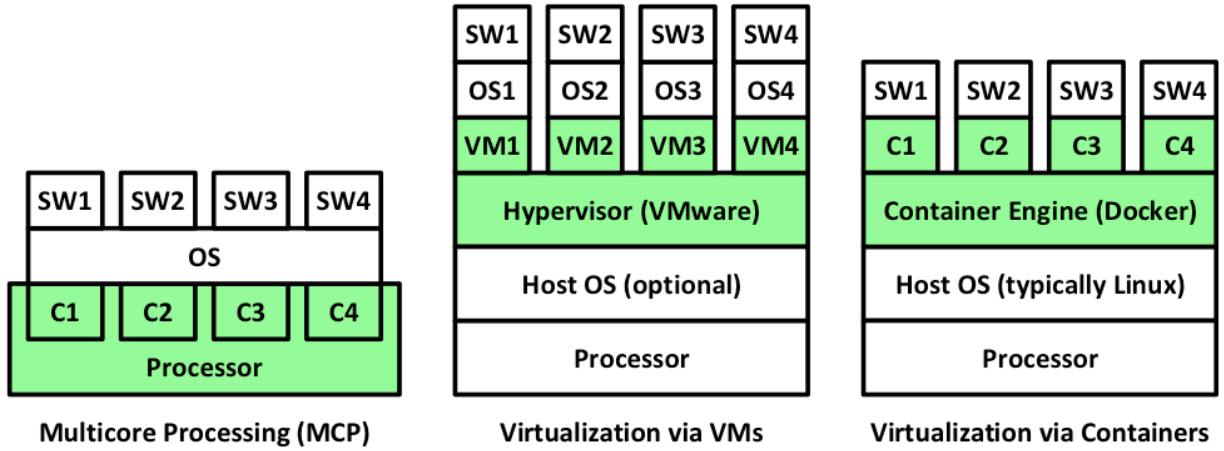
# Virtualization, Virtual Machines, Containers, And Orchestration Systems



# Multicore Processing & Virtualization

📖 *Multicore processing and virtualization underpin modern cloud infrastructure. They allow multiple isolated workloads to share one physical machine. Benefits include cutting costs, improving reliability, and enabling elastic scalability.*

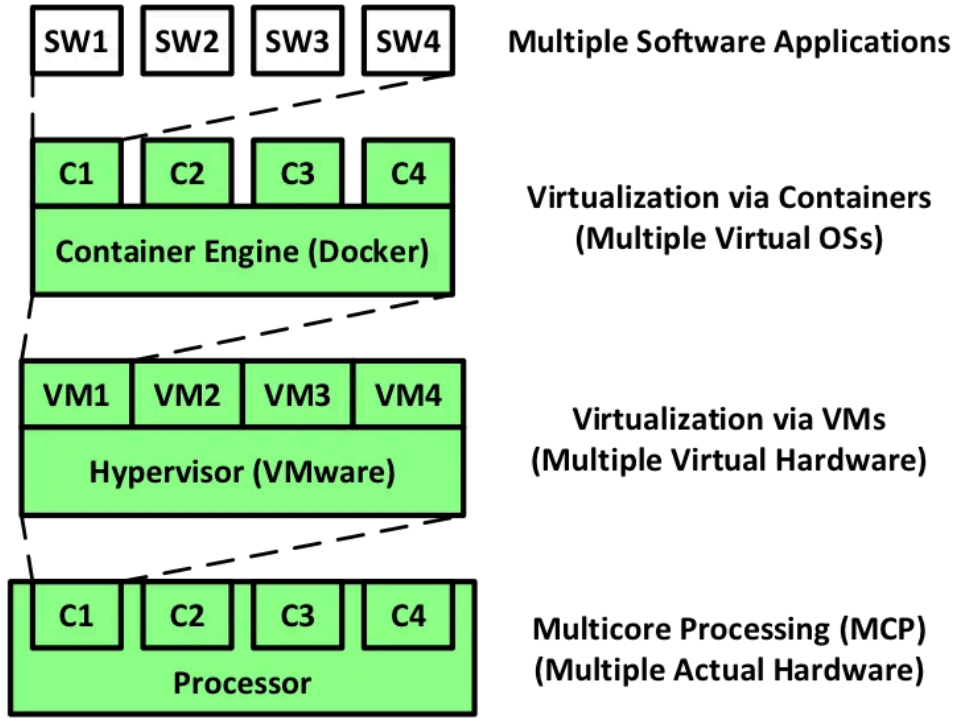
- **Multicore Processing (MCP)**
  - single chip with multiple real cores
  - enables true parallel execution
- **Virtual Machine (VM)**
  - software simulation of hardware
  - runs a full guest OS in isolation
- **Hypervisor (VMM)**
  - supervises guest OS execution on top of physical host hardware
- **Container**
  - virtual runtime environment (OS kernel)
  - no hardware emulation



# Virtualization

📖 *Virtualization creates a software-based version of a physical resource (server, OS, storage, network) → multiple environments share one physical machine.*

- **Guest OS**
  - OS running inside the VM
- **Host OS**
  - OS running the virtualization system
- Encapsulates physical machine features in a virtual environment
- Reduces hardware expenses and improves scalability




*Layered view: Cores → Hypervisor/VMs → Container Engine → Apps*

# Virtualization Evolution

## Stage 1: Pre-Virtualization

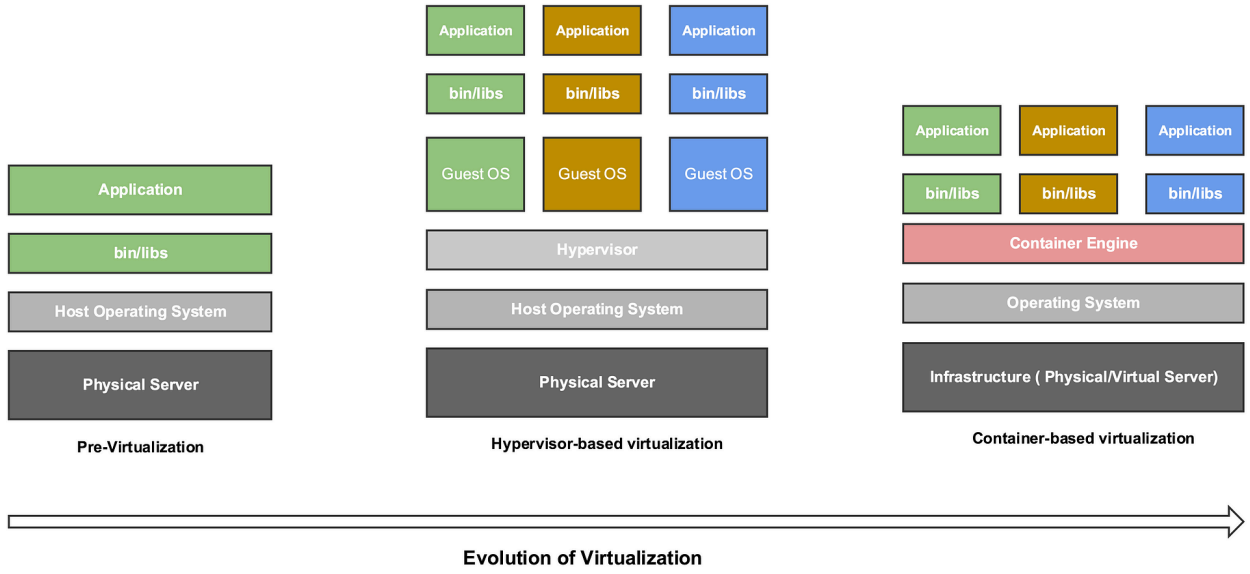
- One app per physical server
- **Problems**: high cost, slow deployment, hard to migrate

## Stage 2: Hypervisor-based

- Multiple VMs per host
- **Benefits**: cost-efficient, easy to scale
- **Problems**: kernel duplication, portability issues

## Stage 3: Container-based

- Container engine on top of OS
- **Benefits**: runtime isolation, portability, fast boot, cost-efficient



# Types Of Virtualization

- **Hardware / Server Virtualization**
  - hypervisor creates multiple VMs on one physical server (VMware ESXi, Hyper-V)
- **Software Virtualization**
  - OS or app runs on a platform different from its native one (emulators, BlueStacks)
- **Desktop Virtualization (VDI)**
  - desktop environment hosted on a remote server; user accesses via any device
- **Storage Virtualization**
  - multiple physical storage devices unified into one logical pool; centralized management
- **Network Virtualization (SDN/NFV)**
  - physical network resources abstracted into virtual networks (VLANs, overlays)

**Containers** are a form of **OS-level virtualization**

- virtualize the operating system, not the hardware.
- distinct category from hypervisor-based VM virtualization.

# Hypervisors

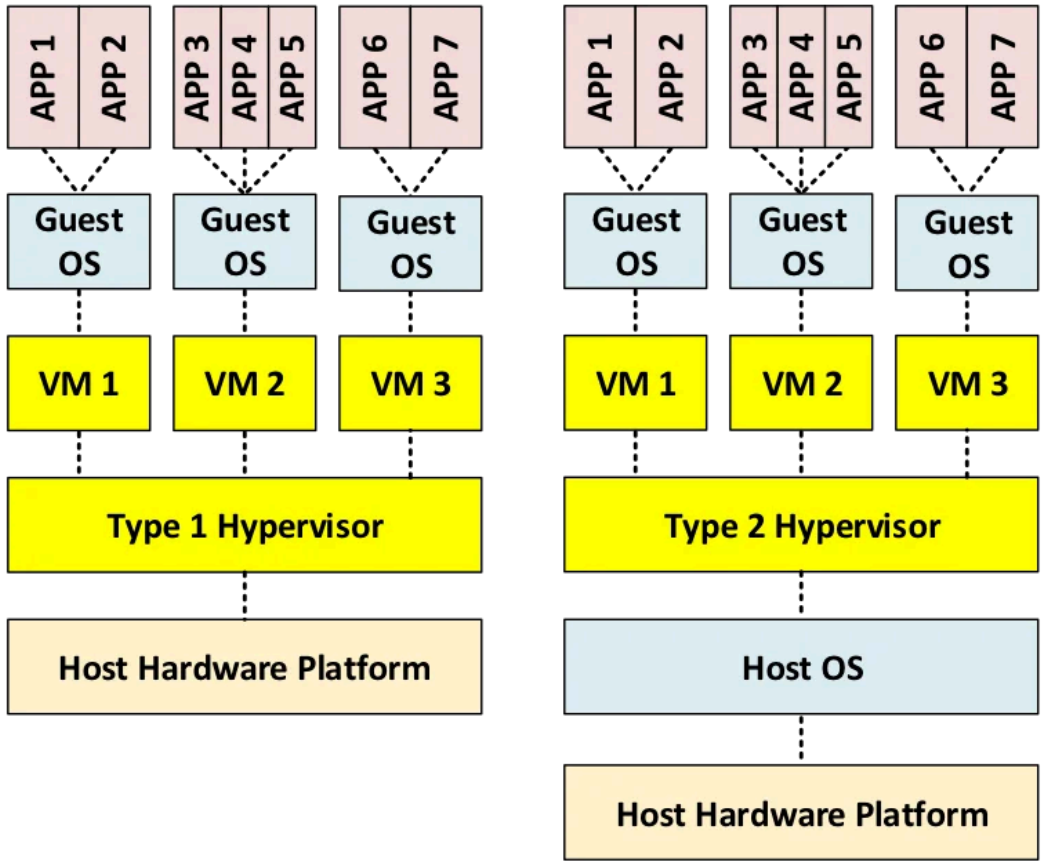
📖 *A hypervisor (VMM — Virtual Machine Monitor) abstracts physical hardware to create and manage multiple VMs. Type 1 runs directly on bare metal; Type 2 runs as an application on a host OS.*

## Type 1 (Bare Metal)

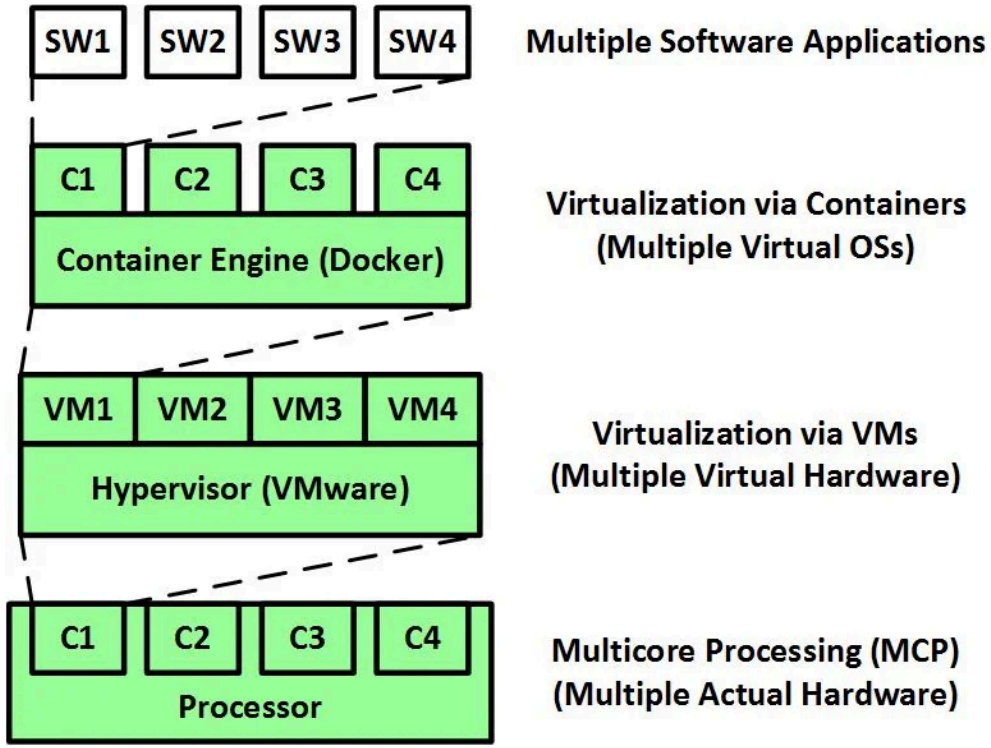
- Runs directly on hardware
- Examples: VMware ESXi, Proxmox, Xen, KVM, Hyper-V

## Type 2 (Hosted)

- Runs as an application on existing OS
- Examples: VirtualBox, VMware Workstation, Parallels, QEMU, UTM



# Virtualization Layers

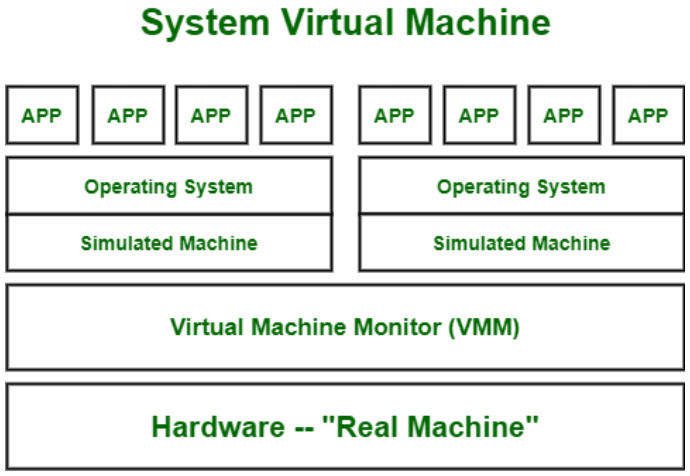



> Abstraction layers: Hardware (Processor) → VM virtualization (Hypervisor + VMs) → OS virtualization (Container Engine + Containers) → applications.
> Each layer adds isolation and portability.

# Virtual Machines

## System Virtual Machine

- Complete platform — full OS execution
- Hardware distributed between simulated OSs
- Multiple different OSes can run simultaneously
- Example: VirtualBox, VMware, Hyper-V




## Process Virtual Machine

- Runs a single application or process
- Provides a platform-independent runtime environment
- Environment destroyed on exit — no state persists
- Example: **JVM** (Java), **Wine** (run Windows apps on Linux)

**Virtual Machine Language**: Platform-independent bytecode executed by the VM runtime.

- The VM translates bytecode to native machine code at runtime (JIT).
- why Java runs identically on Windows, Linux,...

# Use Cases For VMs

- **Isolation & Sandboxing** → Running untrusted code
- **Resource Allocation** → Limit max resources per app/user
- **System Management** → Easy backup/restore entire VM
- **Cross-platform Testing** → Test apps on other OSs
- **Prototyping** → Virtualize entire networks (GNS3)
- **Emulation** → Run legacy applications, different CPU architectures
- **Just for fun** → Try Linux without changing your computer

# VM Pros & Cons

## Pros

- **Strong spatial isolation** → each VM has its own memory, storage, and network stack — fully separated from other VMs
- **Hardware portability** → run software for different/older OS and hardware
- **Cost** → consolidate many lightly-loaded servers into one
- **System management** → live migration, snapshots, centralized tooling
- **Disaster recovery** → clone, restore, failover
- **Flexibility** → multiple OS families on same physical host
- **Security** → small Type-1 hypervisor attack surface; compromised VM is replaced from clean image

## Cons

- **Resource overhead** → each VM carries full guest OS (GBs of RAM/disk)
- **Shared resources** → hypervisor, L3 cache, memory bus are single points of failure
- **Interference** → apps in different VMs can affect each other's timing (temporal isolation not guaranteed)
- **Slow startup** → boot full OS per VM (seconds to minutes)
- **VM escape** → vulnerability where attacker breaks out of VM into hypervisor or another VM
- **Real-time unsuitability** → jitter from hypervisor scheduling breaks hard deadlines
- **Licensing cost** → commercial hypervisors add cost

# Emulation Vs Virtualization

## Emulation

- Recreates different CPU architectures
- Mimics hardware in software
- Slower but more flexible
- Example: Running console games, running ARM apps on x86





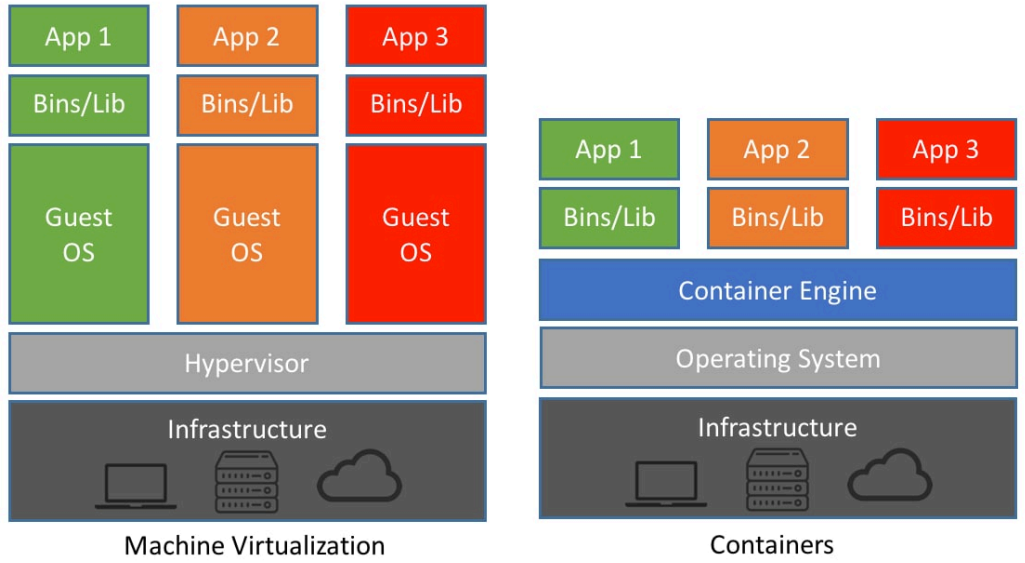
## Virtualization

- Works when OS/software is written for the same hardware class
- Uses hardware directly — no translation layer
- Much faster than emulation
- Requires CPU extensions: Intel VT-x, AMD-V

# Containers

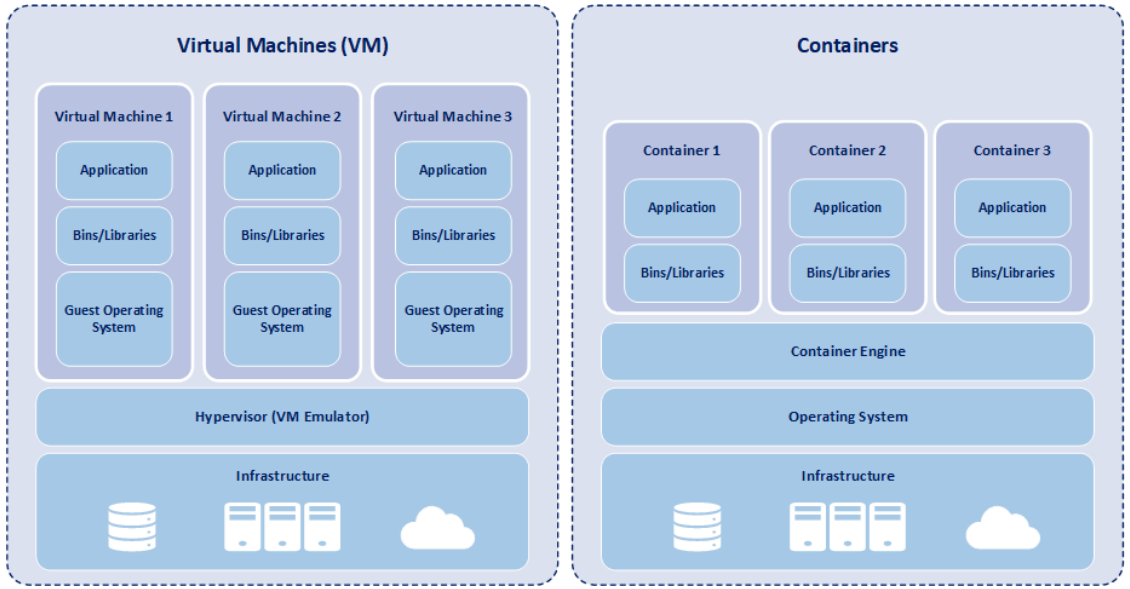
> 📖 *A container is a virtual runtime environment that runs on top of a single OS kernel → virtualizes the operating system (not the hardware)*

- Lightweight, portable execution environments
- Share host OS kernel — no full guest OS needed
- Package code + all dependencies together
- Run hundreds on one machine (vs ~4 VMs)
- Designed for **ephemerality** — stateless by default
- Persistent data stored in **volumes**



# Container Vs Virtual Machine

| Feature | Virtual Machines | Containers |
|---|---|---|
| Virtualizes | Physical infrastructure | Operating system |
| OS per unit | Full guest OS | Share host kernel |
| Size | Large (GBs) | Small (MBs) |
| Boot time | Seconds–minutes | Milliseconds–seconds |
| Isolation | Hardware-level (strong) | Process/namespace (good) |
| Scalability | High cost | Highly scalable |
| Portability | Limited (OS-tied) | True portability |
| Use case | Legacy apps, strict isolation | Microservices, CI/CD |



# Container Pros & Cons

## Pros

- **Hardware cost** → consolidate many apps on fewer machines
- **Performance** → no hardware emulation overhead
- **Scalability** → engine manages large numbers of containers
- **Portability** → same image: dev → test → prod → cloud
- **CI/CD** → fast builds, trivial rollbacks (redeploy old image)
- **Security** → compromised container? Stop + restart from clean image
- **Fast boot** → milliseconds vs minutes for VMs

## Cons

- **Shared kernel** → kernel vulnerability affects all containers on host
- **Weaker isolation** → not suitable for untrusted multi-tenant workloads without extra hardening
- **Container sprawl** → excessive containers are hard to manage
- **Persistent storage** → ephemeral by default; needs volumes for stateful apps
- **Security requires effort** → non-root users, read-only FS, network policies needed

# Container Security

## Isolation Limits

- Containers share the host kernel → a kernel exploit affects all
- Namespaces isolate processes but not the kernel itself
- **Rootless containers**: run container engine without root privileges (Podman, rootless Docker)
- **Read-only filesystem**: `docker run --read-only` prevents writes to container FS

## Image Hardening

- Never use `latest` in production → pin to digest: `image@sha256:...`
- **Image scanning**: Trivy, Snyk, Docker Scout detect CVEs before deploy

## Runtime Controls

- **seccomp**: restrict system calls the container can make
- **AppArmor / SELinux**: mandatory access control profiles
- **Capabilities**: drop all, add only what's needed (`--cap-drop ALL --cap-add NET_BIND_SERVICE`)

> Run containers as **non-root** (`USER appuser` in Dockerfile). Most images default to root → this is a security risk in production.

# Container History

> Process isolation in Linux evolved over 40+ years before Docker.

## Timeline

- **1979** `chroot`: first process isolation → change root directory of a process
- **2000** FreeBSD **jails** → stronger isolation, own network/IP
- **2001** Linux-VServer → partition filesystem, network, memory
- **2005** OpenVZ → OS-level virtualization for Linux
- **2008 LXC** (Linux Containers) → first complete container manager using cgroups + namespaces
- **2013 Docker** → built on LXC initially

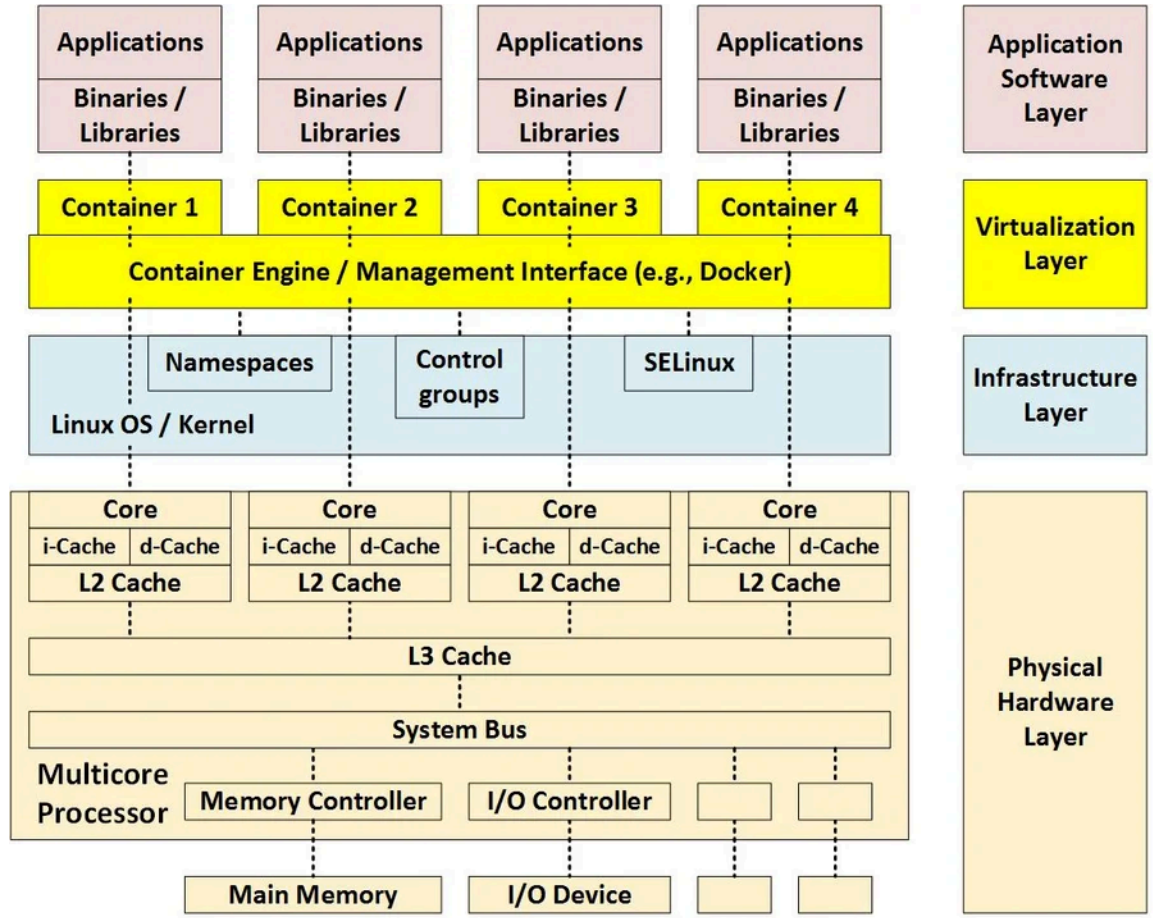
## Docker's Key Innovation

- Started using LXC, then replaced it with **libcontainer** (written in Go, 2014)
- libcontainer gave Docker direct control over namespaces, cgroups, AppArmor, network → without depending on LXC version
- Added: **image layering**, **Docker Hub**, **portability** → what LXC lacked

> Docker didn't invent containers. It made them **usable by everyone** → simple build, ship, run workflow with a public registry.

# Container Architecture

- Containers are ordinary OS **processes** → isolated via Linux kernel features (namespaces, cgroups, UnionFS), not emulated.
- All containers on a host share the **same kernel**.
- The container engine (Docker, containerd, CRI-O) automates the kernel configuration and manages the full container lifecycle.



# Container Internals → Namespaces

📖 *A Linux namespace partitions kernel resources so each container sees its own isolated view → processes, network, filesystem, users, and hostname scoped per container.*

- **PID** → independent process tree; container's first process is PID 1
- **Network** → own routing table, IP addresses, sockets
- **Mount** → independent filesystem mount points
- **User** → own user/group IDs (root inside ≠ root outside)
- **IPC** → isolated shared memory and message queues
- **UTS** → own hostname and domain name

- Namespaces in the Linux kernel since **2002**. Container support became practical in **2013** with Docker.
- A process belongs to **multiple namespaces** simultaneously → one PID, one network, one mount, etc.
- Parent namespaces see into child namespaces (not vice versa).
- The host can inspect container processes; containers cannot see the host's process tree.

# Container Internals → Cgroups, UnionFS & Engine

## cgroups → Resource Control

Limit and account for resource usage per process group:

- **Limits** → max CPU time, memory, I/O bandwidth
- **Prioritization** → weight resources between competing groups
- **Accounting** → monitor usage per cgroup
- **Control** → freeze, stop, restart all processes at once

> In Kubernetes, cgroups enforce **resource requests & limits** at the Pod level → the scheduler uses them to bin-pack workloads efficiently.

## UnionFS → Layered Filesystem

Read-only image layers + one writable layer on top.
Layers shared → efficient storage, fast startup.
Common implementations: **overlay2**, zfs, btrfs, aufs

## Container Engine

Automates namespace/cgroup setup and manages full lifecycle (pull, create, start, stop, remove).

| Engine | Notes |
|---|---|
| **Docker** | Developer UX, CLI, Compose |
| **containerd** | K8s default runtime |
| **CRI-O** | Lightweight K8s runtime |
| **Podman** | Rootless, daemonless |

# OCI — Open Container Initiative

📖 *OCI standardizes container image format and runtime so any compliant tool can build and run containers interchangeably.*

## The Runtime Stack

- `runc` → OCI reference runtime
  - low-level: talks directly to Linux kernel (namespaces, cgroups)
- `containerd` → manages image pull, storage
  - calls `runc` to start containers
- **Docker** → adds developer UX (CLI, build, Compose) on top of `containerd`
- **Podman / CRI-O** → alternative container engines, also OCI-compliant

## Why it Matters

Before OCI (2015), Docker's format was proprietary. OCI decoupled:

- **Image spec**: how an image is packaged (layers, manifests)
- **Runtime spec**: how a container is started

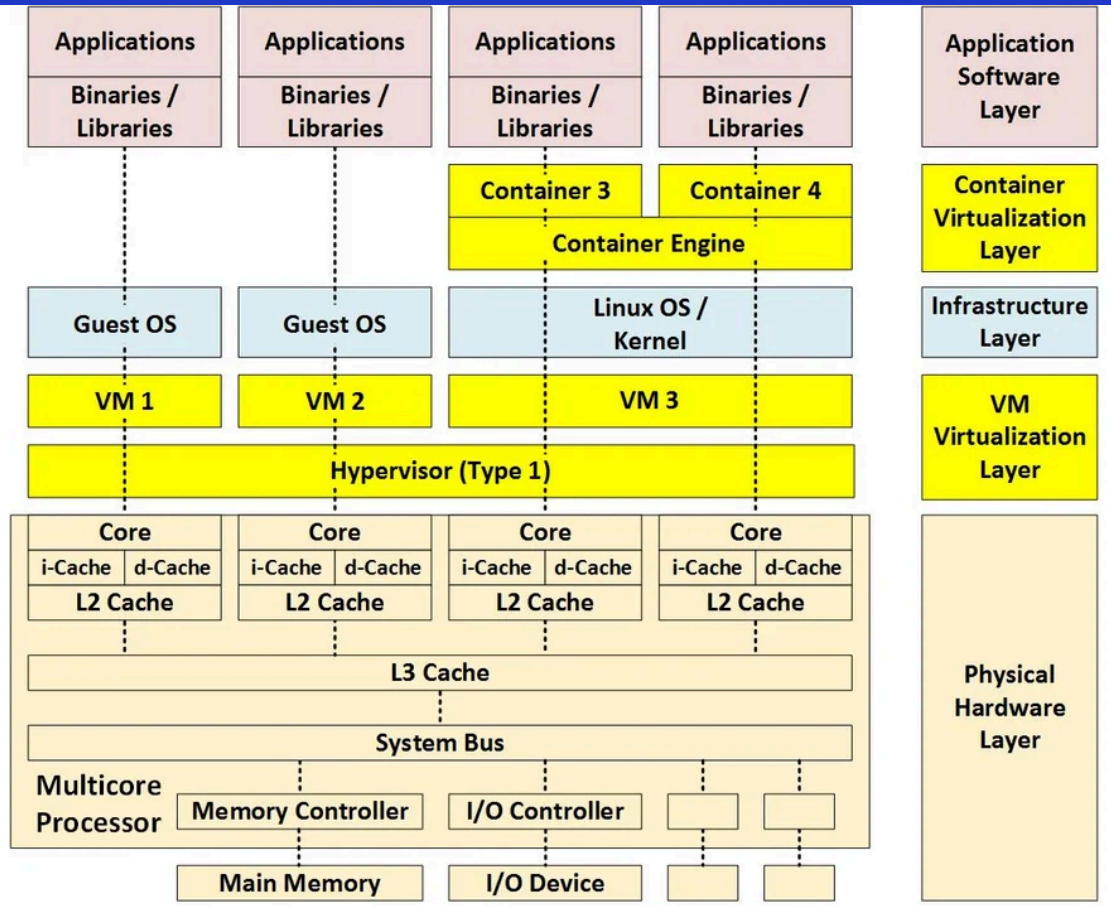
# Hybrid Container Architecture

## Containers on Bare Metal

- Container engine runs directly on host OS
- Maximum performance
- Less isolation between tenants

## Containers on VMs (Hybrid)

- Container engine runs inside a VM
- **Common in cloud** (EC2 + Docker, Azure VM + containers)
- Combines VM strong isolation with container agility




*Most cloud deployments use this model*

- *VM-level isolation + container-level density & speed.*

# Container Filesystem Layers

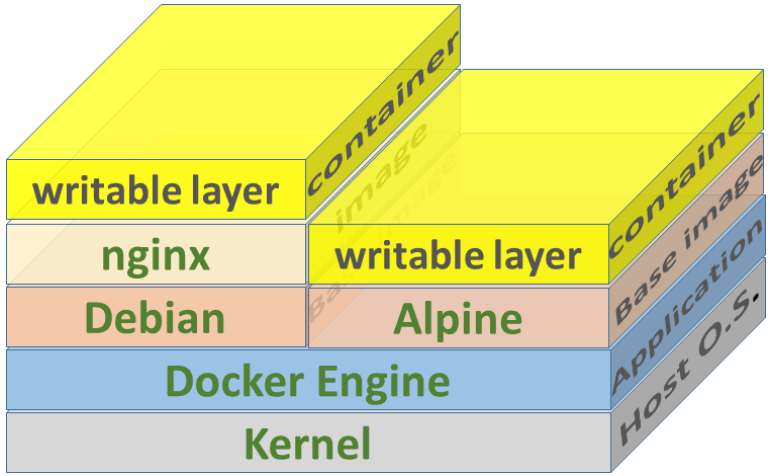
## How Image Layers Work

- Each `Dockerfile` instruction creates one **read-only layer**
- Layers are **shared** across images that use the same base
- Running container adds a thin **writable layer** on top
- Write operations use **Copy-on-Write** (CoW)

```
FROM ubuntu:22.04       # layer 1 (base)
RUN apt-get update      # layer 2
RUN apt-get install -y python3  # layer 3
COPY app.py /app/       # layer 4
CMD ["python3", "/app/app.py"]  # metadata
```

## Benefits

- **Caching**: unchanged layers reused across builds → fast rebuilds
- **Efficiency**: shared base layers not duplicated on disk
- **Auditability**: each layer traceable to a Dockerfile instruction




`docker history myimage` → inspect all layers and sizes

# Container Storage

## Volumes

- Managed by Docker
- Stored → `/var/lib/docker/volumes/`
- **Survive** container lifecycle
- Shareable across containers
- Best for production data

```
docker volume create mydata
docker run -v mydata:/data img
```

## Bind Mounts

- Map a **host path** into container
- Modified by host processes or Docker
- Ideal for development (live code reload)

```
docker run -v $(pwd):/app \
  python:3.11 python app.py
```

## tmpfs Mounts

- **In-memory only** → never written to disk
- Destroyed when container stops
- Use for sensitive data or high-speed temp files

```
docker run
  --tmpfs /tmp:size=100m \
  myimage
```

## Network Drivers

Container networking provides isolation between containers via **network drivers**:

| Driver | Description | Use Case |
|---|---|---|
| **bridge** | Default virtual network on host | Containers on same host |
| **host** | Share host network stack | High-performance, no isolation |
| **overlay** | Multi-host networking | Docker Swarm / Kubernetes |
| **none** | All networking disabled | Maximum isolation |
| **macvlan** | Container gets its own MAC/IP | Legacy apps needing L2 access |

```
docker network create mynet
docker run --network mynet --name app myimage
docker run --network mynet --name db postgres  # app can reach db by hostname
```

# Container Network Model (CNM)

📖 *CNM is Docker's standard for container networking. Kubernetes uses CNI (Container Network Interface) instead — both solve the same problem with different APIs.*

## CNM Core Objects

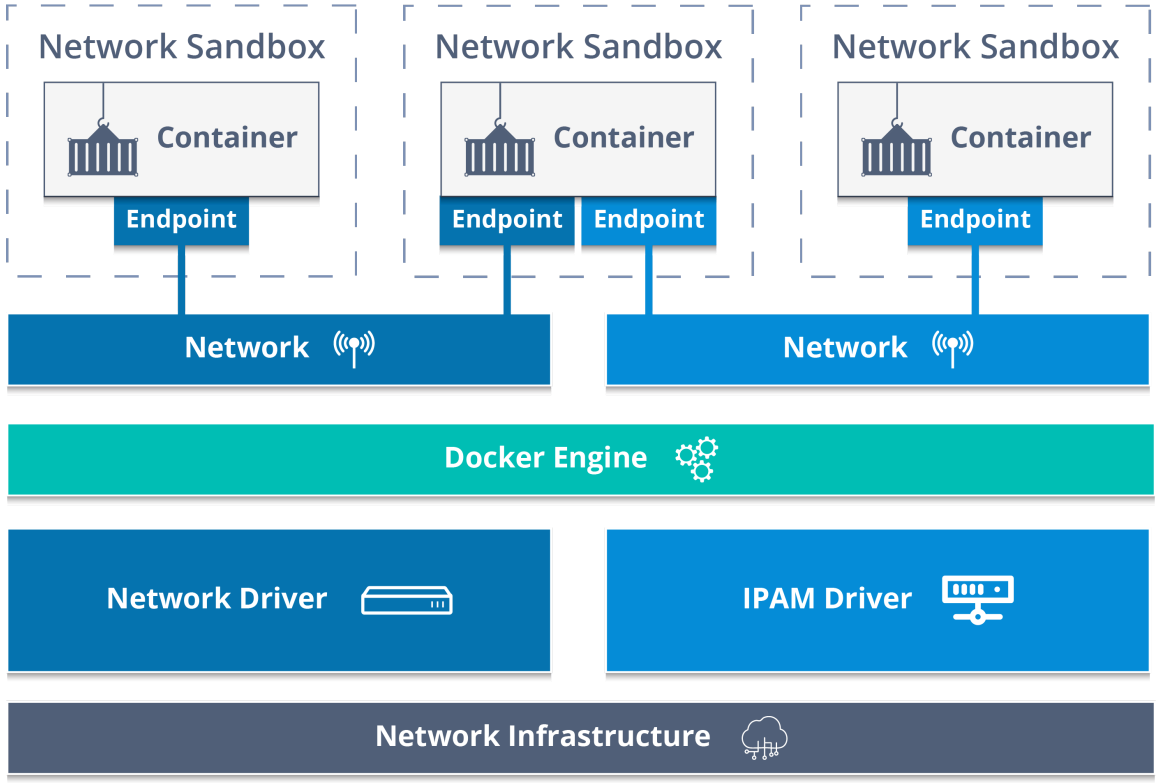
**Sandbox** → container's full network config: IP address, MAC address, routes, DNS

**Endpoint** → connects a sandbox to a network (one per network the container joins)

**Network** → group of endpoints that can communicate; isolated from other networks

**Network Controller** → entry point into libnetwork; attaches drivers to networks

**Driver** → owns and manages the network (bridge, overlay, macvlan…)




Architecture of Container Networking Model

# Monolith Vs Microservices

## Monolithic Architecture

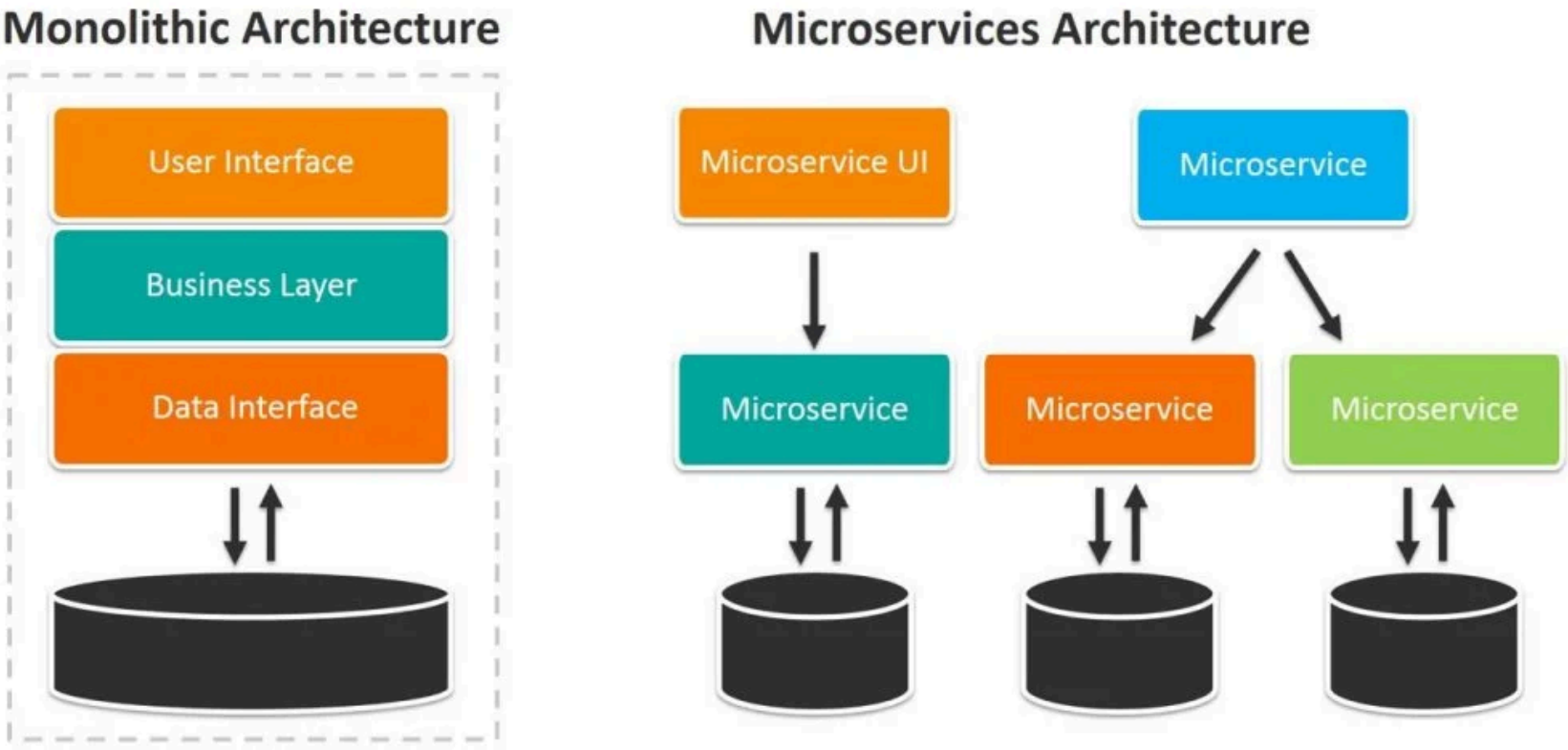
- Single deployable unit → all features in one codebase
- Scale by running more copies of the entire app
- One failure can bring down everything
- Easy to develop initially; painful to maintain at scale

## Microservices Architecture

- Each feature is an **independent service** with its own codebase, database, and deployment
- Services communicate via APIs or message queues
- Scale only the services under load
- Each service runs in its own **container**

**why containers matter ?**: 10 microservices = 10 independent containers.
Orchestration (Kubernetes) manages them at scale (health checks, scaling, rolling updates, service discovery).

# Monolith Vs Microservices — Architecture




> *Monolith: one deployable unit. Microservices: independent services, each in its own container, communicating over APIs.*

# Container Orchestration

📖 *Container orchestration automates deployment, scaling, networking, and management of containerized applications across a cluster of hosts.*

### Why Orchestrate?

- Many containers need coordinated management
- Automated deployment across nodes
- Load balancing across replicas
- Scaling based on demand
- Self-healing (restart failed containers)
- Service discovery between components

# Open-source container orchestration platform

## Origin

- Developed by **Google**
- Evolved from Google's internal **Borg** project (runs billions of containers/week)
- Result of the **microservices** revolution
- Manages containerized apps on physical, virtual, or cloud infrastructure

## Key Features

- High availability (no downtime)
- Scalability (high performance)
- Disaster recovery (backup/restore)
- Service discovery and load balancing
- Automated rollouts and rollbacks
- Automatic bin packing
- Self-healing
- Secret and configuration management

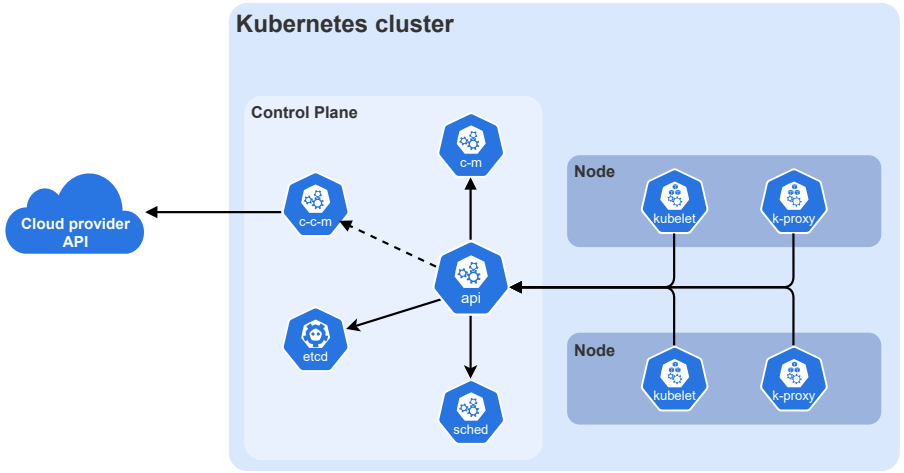
# Kubernetes Architecture

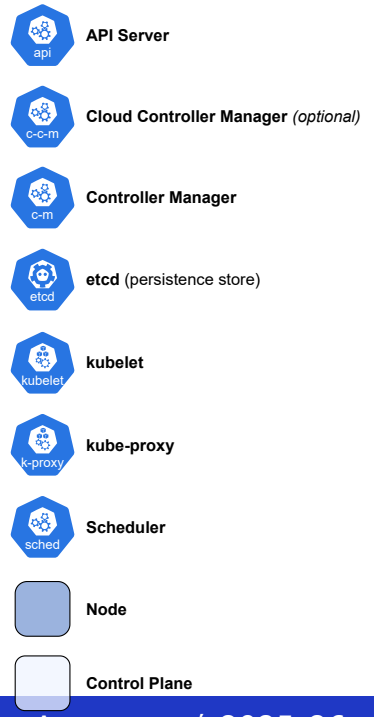
## Control Plane (The Brain)

- **API Server**: Expose Kubernetes API
- **etcd**: Key-value store for cluster data
- **Scheduler**: Schedule pods to nodes
- **Controller Manager**: Node, replication, endpoints controllers

## Worker Nodes (The Muscle)

- **kubelet**: Agent ensuring containers run in a Pod
- **kube-proxy**: Network proxy per node
- **Container Runtime**: containerd, CRI-O *(Docker deprecated since K8s v1.24)*



# Kubernetes Entities

## Workloads

- **Pod**: Smallest deployable unit (1+ containers)
- **Deployment**: Manage stateless pod replicas
- **StatefulSet**: Ordered pods with stable identity (databases)
- **DaemonSet**: One pod per node (logging, monitoring)
- **Job / CronJob**: Run-to-completion and scheduled tasks

## Networking & Storage

- **Service**: Stable endpoint for pods (ClusterIP, NodePort, LoadBalancer)
- **Ingress**: External HTTP/HTTPS routing with rules
- **Volume / PersistentVolume**: Persistent storage beyond pod lifecycle

## Configuration

- **Namespace**: Virtual cluster for isolation
- **ConfigMap / Secret**: App config and sensitive data
- **Labels / Annotations**: Metadata for selection and tooling

# Helm → Kubernetes Package Manager

📖 *Helm → packages Kubernetes manifests into reusable, versioned **charts**.*

## Without Helm

- Deploy an app → write and manage 5+ YAML files:
  - `Deployment.yaml` — how many replicas to run and which image
  - `Service.yaml` → stable network endpoint to reach the pods
  - `ConfigMap.yaml` → non-secret config (env vars, config files)
  - `Ingress.yaml` → HTTP routing rules from outside the cluster
  - `PersistentVolumeClaim.yaml` → request for persistent storage
- Every environment (dev/staging/prod) needs its own copy.

## With Helm

```
# Install nginx with one command
helm install my-nginx bitnami/nginx

# Install with custom values
helm install my-app ./my-chart \
  --set image.tag=v2.1 \
  --set replicas=3

# Upgrade / rollback
helm upgrade my-app ./my-chart
helm rollback my-app 1

# Remove all resources at once
helm uninstall my-app
```

> Artifact Hub hosts public charts (databases, monitoring, CI tools).

# Kubernetes Vs Docker

**Docker → builds the containers.**

- Build, ship, run containers
- Single host focus
- Development / testing / CI pipelines
- Simple: one daemon, one machine
- Manual scaling and failover

**Kubernetes → decides where, when, and how many run.**

- Orchestrate containers at scale
- Multi-node clusters
- Production-grade: HA, self-healing, auto-scaling
- Declarative config (desired state)
- Manages hundreds of Docker containers automatically

> *Docker vs Kubernetes → They are complementary, not competing.*

# Cloud Kubernetes Services

| Service | Provider | Key Differentiator |
|---|---|---|
| **EKS** | Amazon AWS | Deep AWS integration (IAM, VPC, EBS) |
| **AKS** | Microsoft Azure | Free control plane, AD integration |
| **GKE** | Google Cloud | Kubernetes creator; autopilot mode |
| **IKS** | IBM Cloud | Enterprise focus, OpenShift option |

| Service | Provider | Key Differentiator |
|---|---|---|
| **OKE** | Oracle Cloud | Low cost, bare metal nodes |
| **OpenShift** | Red Hat | Enterprise K8s + developer tooling |
| **Tanzu TKG** | VMware | On-prem + multi-cloud, vSphere integration |

- All managed services handle the **control plane** (API server, etcd, scheduler).
- You manage worker nodes and workloads only.

# Alternative Container Engines → LXD Vs Docker

## LXD (Linux Container Hypervisor)

- Developed by **Canonical** (Ubuntu)
- **System containers**→ behaves like a full OS (multiple processes, init system)
- Run entire Linux distributions in a container
- Supports live migration between hosts
- Linux-only; integrates with OpenStack/OpenNebula
- **Use case**: server consolidation, OS-level isolation
- Docker initially used LXC (2013)

## Docker vs LXD

| | Docker | LXD |
|---|---|---|
| Model | Application containers | System containers |
| Processes | 1 per container | Many (full OS) |
| Platform | Linux + Windows | Linux only |
| Portability | High | Low |
| Ease of use | Easy | Harder |
| Ecosystem | Massive (K8s, CI/CD) | Limited |
| Persistent data | Volumes required | Built-in |
| Used by | Spotify, Google, Twitter | Deck-D, GEMServers |

# Summary

## Virtualization Evolution

1. **Virtual Machines** → Full OS isolation, hypervisor based
2. **Containers** → Lightweight, shared kernel, portable
3. **Orchestration** → Automate container management at scale
4. **Kubernetes** → Industry standard orchestration platform

## The Path Forward

- Development →Docker containers
- High-security/isolated testing → VMs
- Production → Kubernetes orchestration
- Cloud → Managed K8s services
- Advanced → **Service mesh** (Istio, Linkerd) for observability, mTLS, traffic control between microservices

# Questions?

## References

- Kubernetes Documentation
- Docker Documentation
- K8s Architecture
- Container Networking Model
- LXD Documentation